

EC 504
Fall, 2025
Homework 6

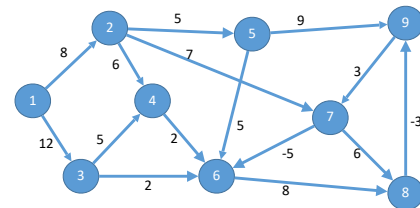
Due Friday, November 21, 2025, 11:59 pm

Problem 5.1 (20 pts) Determine whether the statements below are true or false. Please provide a quick explanation to justify your answer.

- (a) The following three algorithms are examples of greedy algorithms: Dijkstra's shortest path algorithm, Kruskal's minimum spanning tree algorithm, Prim's minimum spanning tree algorithm.
- (b) Bellman-Ford algorithm can be used for finding the longest path in an acyclic graph.
- (c) In Floyd-Warshall's algorithm for finding all-pairs shortest paths, after each major outer loop k , upper bounds are computed on shortest distances $D(i, j)$ between each pair of vertices i, j . These upper bounds correspond to the shortest distance among all paths between vertices i and j which use only intermediate vertices in $1, \dots, k$.
- (d) Consider a directed, weighted graph where every arc in the graph has weight 100. For this graph, executing Dijkstra's algorithm to find a shortest path tree starting from a given vertex is equivalent to performing depth first search.
- (e) In Floyd-Warshall's algorithm, the worst case complexity does not depend on the number of edges.
- (f) Floyd-Warshall's algorithm fails to converge when there are negative weight edges, but no negative weight cycles, in the graph.
- (g) Consider a directed, capacitated graph, with origin vertex s and destination vertex t . The maximum flow which can be sent from s to t is less than or equal to the minimum of the following two quantities: the sum of the capacities of the edges leaving s , and the sum of the capacities of the edges entering t .
- (h) Consider a capacitated, directed graph, with an origin vertex s and a destination vertex t . Then, the maximum flow from vertex s to vertex t must be greater than or equal to the smallest capacity of any edge going into vertex t .
- (i) In the Ford-Fulkerson algorithm with integer capacities, there is a sequence of augmenting paths that carry flow from the origin s to the destination t . The amount of flow carried by each augmenting path can be no greater than the amount of flow carried by the preceding augmenting path.
- (j) The Edmonds-Karp variation of the Ford-Fulkerson algorithm has complexity which is polynomial in the number of edges in the graph.

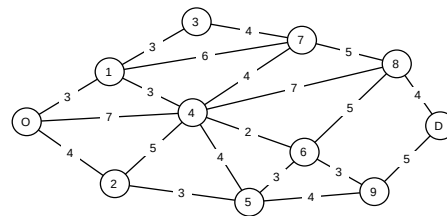
Problem 5.2 (20 pts)

Consider the directed weighted graph on the right. Show the distance estimates computed by each step of the Bellman-Ford algorithm for finding a shortest path tree in the graph, starting from vertex 1 to all other vertices. When scanning edges out of a vertex, consider them in clockwise order starting from straight up. Show the algorithm using a work queue to determine which vertices to scan, in order to avoid unnecessary iterations.



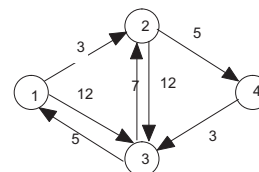
Problem 5.3 (15 pts)

Consider the weighted, undirected graph shown on the right. Assume that the edges can be traveled in both directions with the same distance. Illustrate the steps of Dijkstra's algorithm for finding a shortest path from vertex O to vertex D.



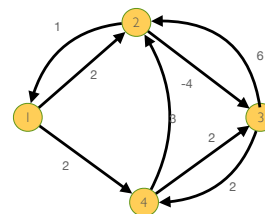
Problem 5.4 (10 points)

Consider the graph indicated on the right. This is a directed graph. Use the Floyd-Warshall algorithm to find the shortest distance for all pairs of vertices. Show your work as a sequence of 4 by 4 tables.



Problem 5.5 (10 points)

Consider the graph indicated on the right. This is a directed graph. Note the negative weight edge from 2 to 3. Use the first step of Johnson's algorithm, to find a reweighing of the edges such that the resulting edge weights are all non-negative. Note that you don't have to follow this with using Dijkstra's algorithm to find the shortest distance for all pairs of vertices; you just have to compute the reweighed distances.

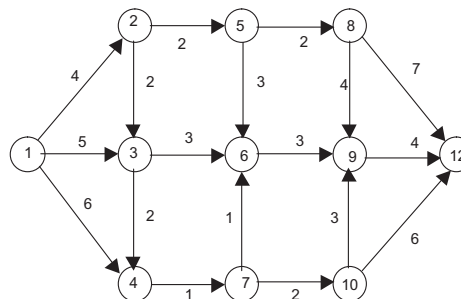


Problem 5.6 (10 pts)

In city streets, the length of an edge often depends on the time of day. Suppose you have a directed graph of streets connecting vertices that represent intermediate destinations, and you are given the travel time on the edge as a function of the time at which you start to travel that edge. Thus, for edge e , you are given $d_e(t)$, the time it takes to travel edge e if you start at time t . These travel times must satisfy an interesting ordering property: You can't arrive earlier if you started later. That is, if $s < t$, then $s + d_e(s) \leq t + d_e(t)$. Suppose that you start at time 0 at the origin vertex 1. Describe an algorithm for computing the minimum time path to all vertices when travel times on edges are time dependent and satisfy the ordering property.

Problem 5.7 (15 pts)

Consider the graph in the figure on the right as a directed, capacitated graph, where the numbers indicate an arc's capacity to carry flow from node 1 to node 12. In the max-flow algorithm of Ford and Fulkerson, the key step is, once a path has been found, to augment the flow and construct the residual graph for the next iteration.



- Consider the path $1 \rightarrow 2 \rightarrow 3 \rightarrow 6 \rightarrow 9 \rightarrow 12$. What is the capacity of this path?
- Suppose we send two units of the flow along path $1 \rightarrow 2 \rightarrow 5 \rightarrow 8 \rightarrow 12$. Construct the residual graph which remains after this flow has been sent. Draw this graph.
- Find the maximum flow from node 1 to node 12 in this graph.